

Java Assignment 6 – Polymorphism

Part A – Method Overloading

Problem 1 – Electricity Bill

Create a Java program to demonstrate **Method Overloading** using an `ElectricityBill` class.

Properties

- `consumerNo`
- `consumerName`
- `unitsConsumed`

Create a constructor to initialize these properties.

Create overloaded methods named `calculateBill()`:

```
calculateBill()  
calculateBill(double rate)  
calculateBill(double rate, int units)
```

Requirements

- `calculateBill()` should calculate the bill using a default rate of ₹8 per unit.
- `calculateBill(double rate)` should calculate the bill using the given rate and the `unitsConsumed` property.
- `calculateBill(double rate, int units)` should calculate the bill using the given rate and the specified number of units.

Display the bill amount calculated using each overloaded method.

2. Online Shopping – Calculate Product Price

Problem Statement

Create a Java program to demonstrate **Method Overloading** using a `Product` class.

Create the following properties:

- `productId`
- `productName`

- price

Create a constructor to initialize the properties.

Create overloaded methods named `calculatePrice()`:

```
calculatePrice()  
calculatePrice(int quantity)  
calculatePrice(int quantity, double discount)
```

Requirements

- `calculatePrice()` should return the price of one product.
 - `calculatePrice(int quantity)` should calculate the total price based on quantity.
 - `calculatePrice(int quantity, double discount)` should calculate the total price after applying the given discount percentage.
 - Display the final amount for each method.
-

Part B – Method Overriding

For the following three programs, use **Hierarchical Inheritance**.

The superclass should contain a common method, and each subclass should **override** that method with its own implementation.

3. Bank Account – Interest Calculation

Problem Statement

Create a Java program to demonstrate **Method Overriding** using hierarchical inheritance.

Create a superclass `BankAccount`.

Superclass – `BankAccount`

Properties:

- `accountNo`
- `accountHolderName`
- `balance`

Create a constructor to initialize these properties.

Create the method:

```
calculateInterest()
```

Provide a general implementation in the superclass.

Subclass – SavingsAccount

Additional property:

- `interestRate`

Override:

```
calculateInterest()
```

to calculate interest according to the savings account interest rate.

Subclass – CurrentAccount

Additional property:

- `interestRate`

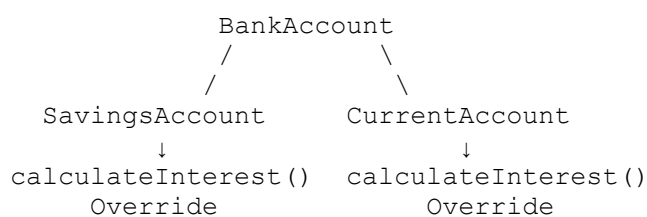
Override:

```
calculateInterest()
```

to calculate interest according to the current account rules.

Create objects for `SavingsAccount` and `CurrentAccount` and display the calculated interest.

Inheritance



4. Hospital Management – Treatment Cost

Problem Statement

Create a Java program to demonstrate **Method Overriding** using hierarchical inheritance.

Create a superclass `Patient`.

Superclass – Patient

Properties:

- `patientId`
- `patientName`
- `age`

Create a constructor to initialize these properties.

Create the method:

```
calculateTreatmentCost()
```

The superclass should provide a general treatment cost calculation.

Subclass – InPatient

Additional properties:

- `numberOfDays`
- `roomCharge`

Override:

```
calculateTreatmentCost()
```

to calculate the treatment cost including room charges based on the number of days.

Subclass – OutPatient

Additional properties:

- `consultationFee`
- `medicineCost`

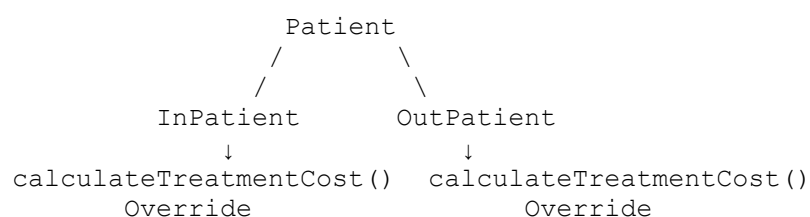
Override:

```
calculateTreatmentCost()
```

to calculate the treatment cost including consultation and medicine charges.

Create objects for both `InPatient` and `OutPatient` and display their respective treatment costs.

Inheritance



5. Vehicle – Rental Charge Calculation

Problem Statement

Create a Java program to demonstrate **Method Overriding** using hierarchical inheritance.

Create a superclass `Vehicle`.

Superclass – `Vehicle`

Properties:

- `vehicleNo`
- `brand`
- `baseRate`

Create a constructor to initialize these properties.

Create the method:

```
calculateRental()
```

The superclass should provide a general rental calculation.

Subclass – `Car`

Additional properties:

- `numberOfDays`
- `insuranceCharge`

Override:

```
calculateRental()
```

to calculate the rental amount based on the number of days and insurance charge.

Subclass – `Bike`

Additional properties:

- `numberOfDays`
- `helmetCharge`

Override:

```
calculateRental()
```

to calculate the rental amount based on the number of days and helmet charge.

Create objects for both `Car` and `Bike` and display their rental charges.

Inheritance

